

스마트 도어락 시 스템

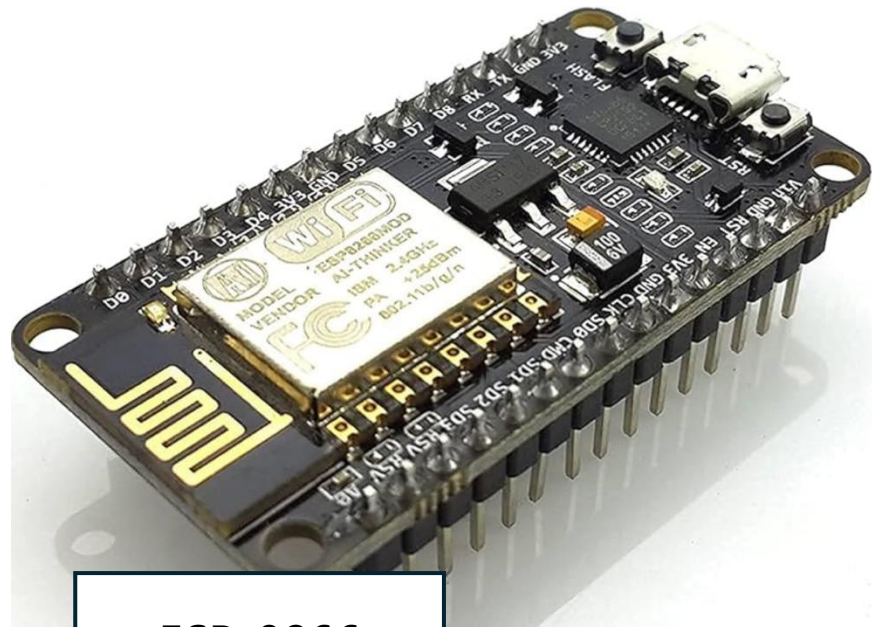
주제 선정 이유

IoT프로세서가 과목명인 만큼, 인터넷을 적극 활용하여 실생활에 도움을 주는 시스템을 만들고자 하였고, 그 중 평소에 유용하게 사용하던 기능을 구현하고자 하였다.

외부에 있는데 집 앞에 중요한 손님이 찾아왔을 때, 문 앞 복도에 CCTV나 많은 사람이 있어 비밀번호 치기 부담스러울 때 등 다양한



준비물

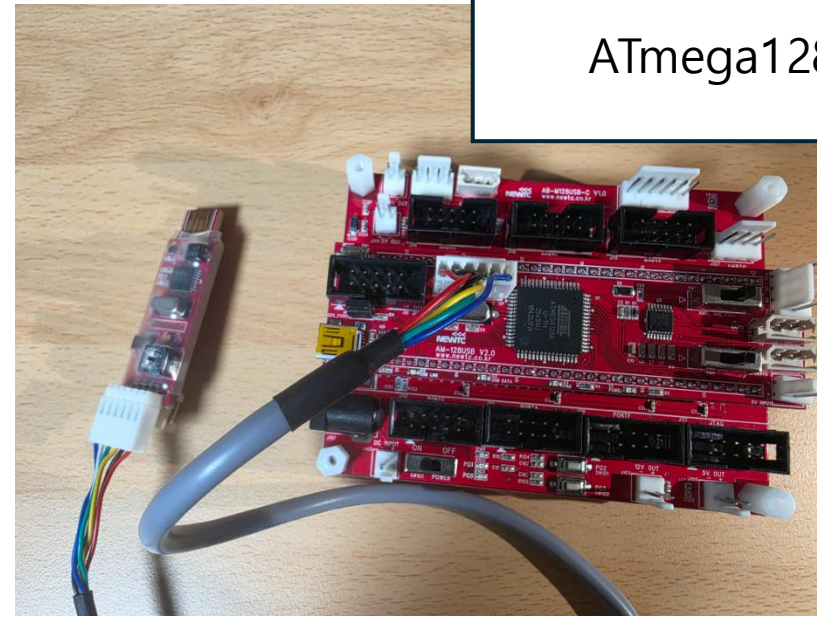


ESP-8266

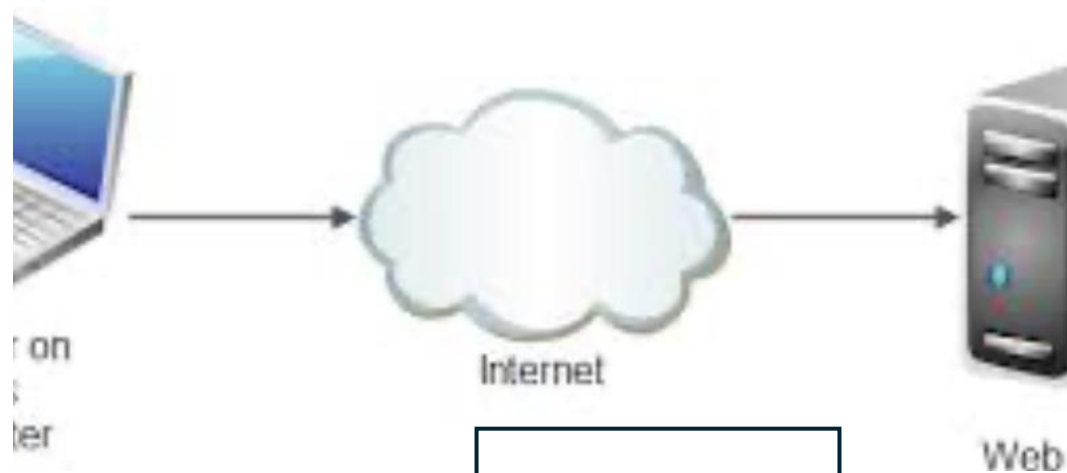
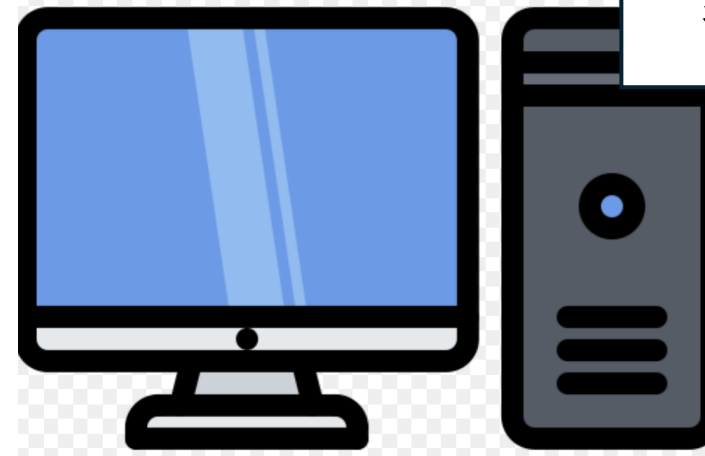
서보모터



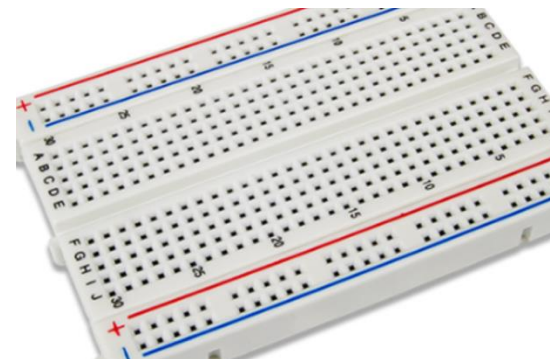
ATmega128

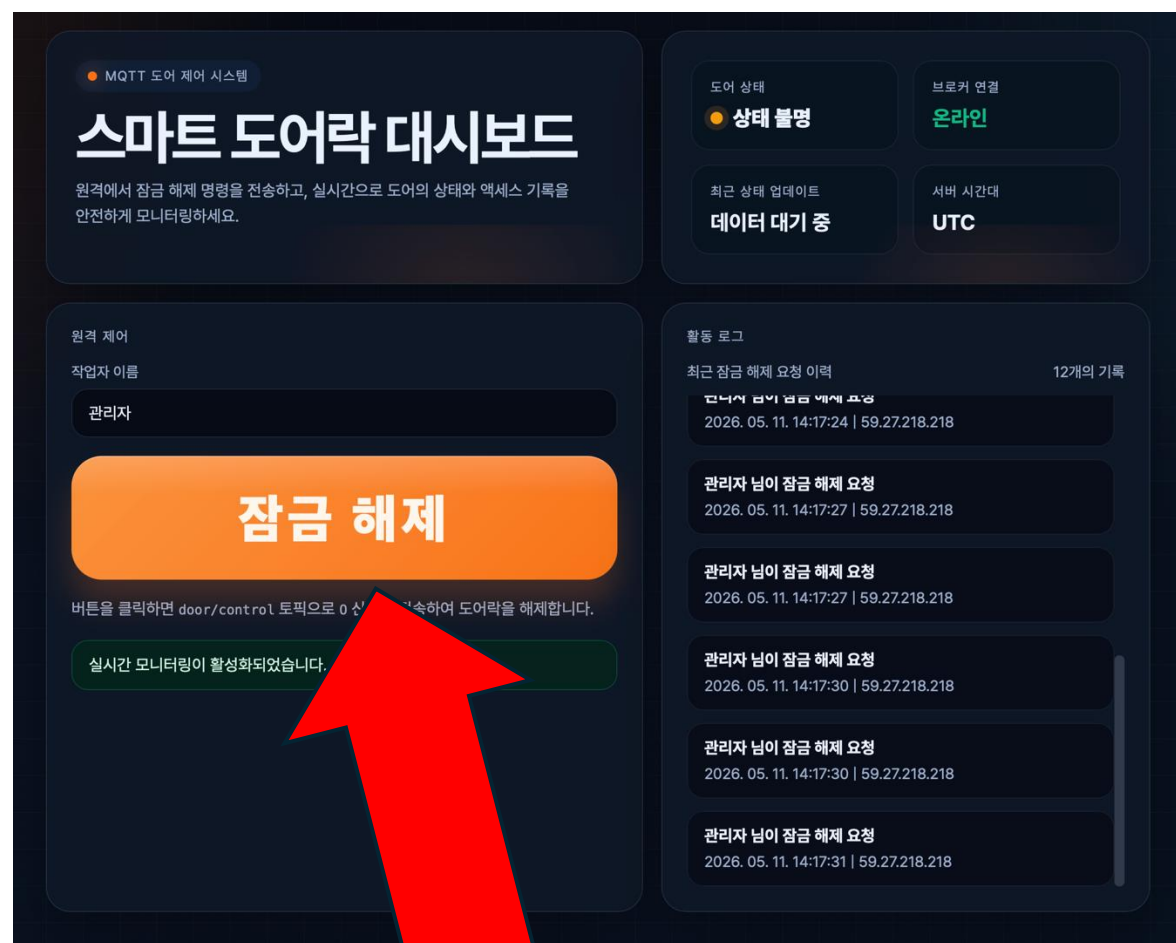


컴퓨터



인터넷/서버





누르면 서보모터
동작

<https://iot.jimindex.com/>

MQTT 도어 제어 시스템

스마트 도어락 대시보드

원격에서 잠금 해제 명령을 전송하고, 실시간으로 도어의 상태와 액세스 기록을 안전하게 모니터링하세요.

도어 상태

상태 불명

브로커 연결

온라인

최근 상태 업데이트

데이터 대기 중

서버 시간대

UTC

원격 제어

작업자 이름

관리자

잠금 해제

버튼을 클릭하면 door/control 토픽으로 0 신호를 전송하여 도어락을 해제합니다.

활동 로그

최근 잠금 해제 요청

관리자님이 잠금 해제 요청

2026. 05. 11. 14:17:27 | 59.27.218.218

관리자님이 잠금 해제 요청

2026. 05. 11. 14:17:27 | 59.27.218.218

관리자님이 잠금 해제 요청

2026. 05. 11. 14:17:30 | 59.27.218.218

관리자님이 잠금 해제 요청

2026. 05. 11. 14:17:30 | 59.27.218.218

관리자님이 잠금 해제 요청

2026. 05. 11. 14:17:31 | 59.27.218.218

Door/control토픽으로 0을 발행하는 코드>>

Door/control 토픽을 구독하고 있는 클라이언트에게 문자 "0"을 발행

```
app.post("/api/unlock", (req, res) => {
  // MQTT 브로커 연결이 없으면 문 열기 명령을 보낼 수 없으므로 바로 종료한다.
  if (!dashboardState.mqttConnected) {
    res.status(503).json({
      ok: false,
      message: "MQTT broker is not connected.",
    });
    return;
  }

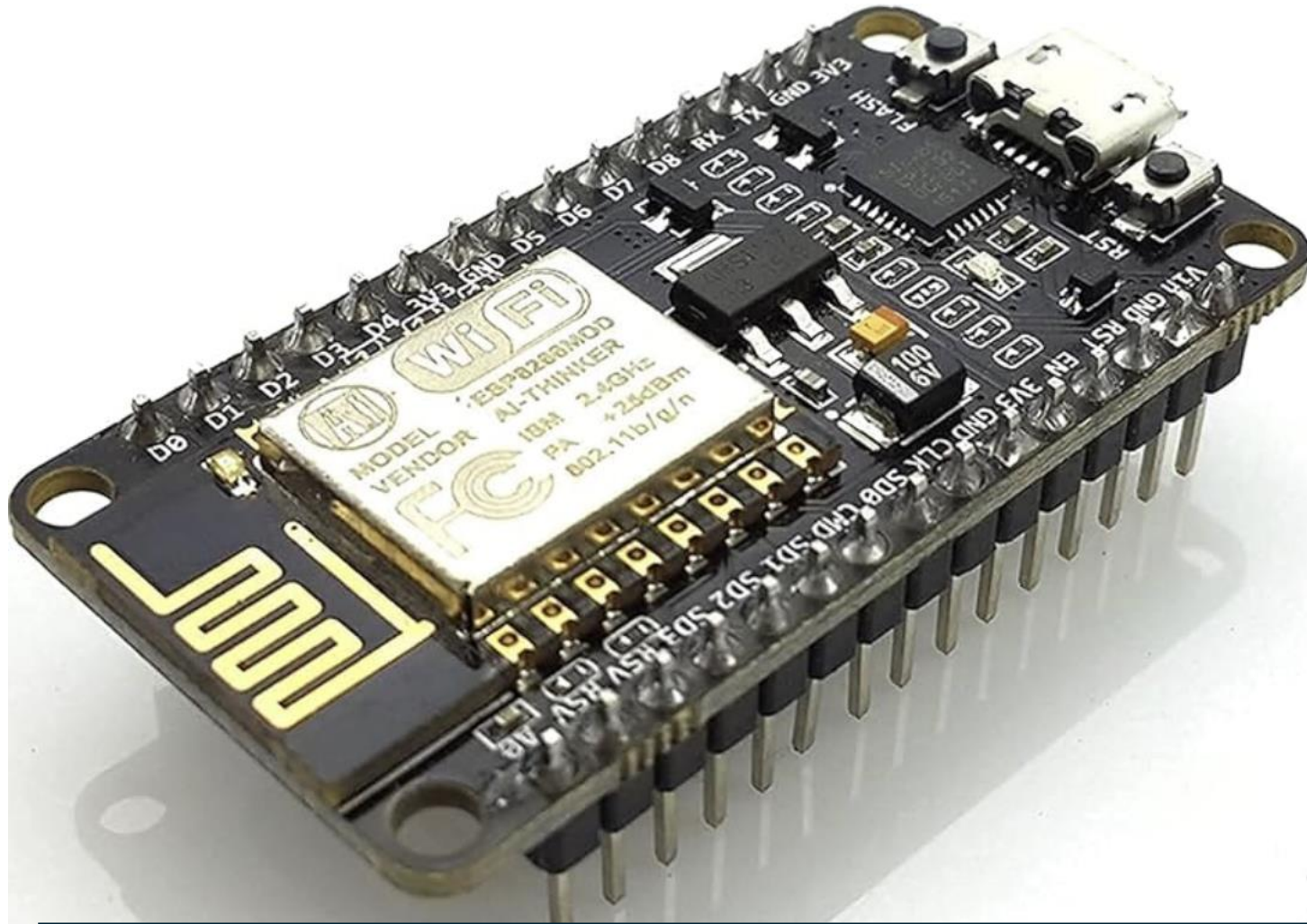
  const actor = sanitizeActor(req.body?.actor);
  const sourceIp = normalizeIp(req.ip);

  // door/control 토픽으로 "0"를 발행하고, 브로커 처리 결과를 콜백에서 확인한다.
  mqttClient.publish(CONTROL_TOPIC, "0", { qos: 1 }, (error) => {
    if (error) {
      res.status(500).json({
        ok: false,
        message: "Failed to publish unlock signal.",
      });
      return;
    }

    // 성공하면 로그를 저장하고, 접속 중인 대시보드에 변경 사항을 실시간으로 전송한다.
    const logEntry = createLogEntry(actor, sourceIp);
    addLog(logEntry);
    broadcastState();

    res.json({
      ok: true,
      message: "Unlock signal published.",
      logEntry,
    });
  });
});
```

ESP-8266 보드



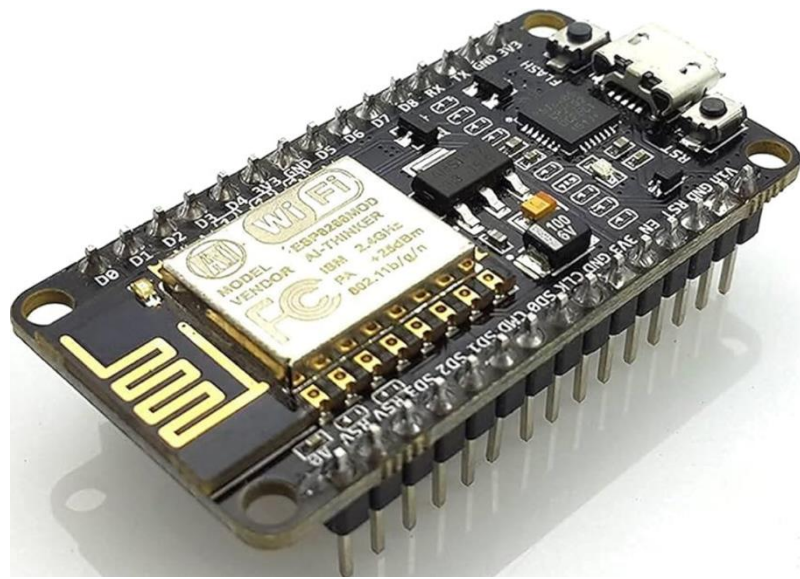
3.3V의 전원으로 동작하는 wifi 기능이 있는 컨트롤러이다.

```
client.setServer("iot.jimindex.com", 1883);
client.setCallback(callback);
client.subscribe("door/control");

void callback(char* topic, byte* payload, unsigned int length) {
    // 수신된 첫 번째 문자가 '0' (Open)인지 확인 [cite: 139, 294]
    if ((char)payload[0] == '0') {
        // ATmega128의 RX(PE0) 핀으로 '0' 문자 전송 [cite: 110, 277]
        Serial.print('0');
    }
}
```

iot.jimindex.com 에서 1883 포트를 통해 door/control 토픽을 구독하고 신호를 받으면 '0'인지 확인하고 Atmega128로 전송.

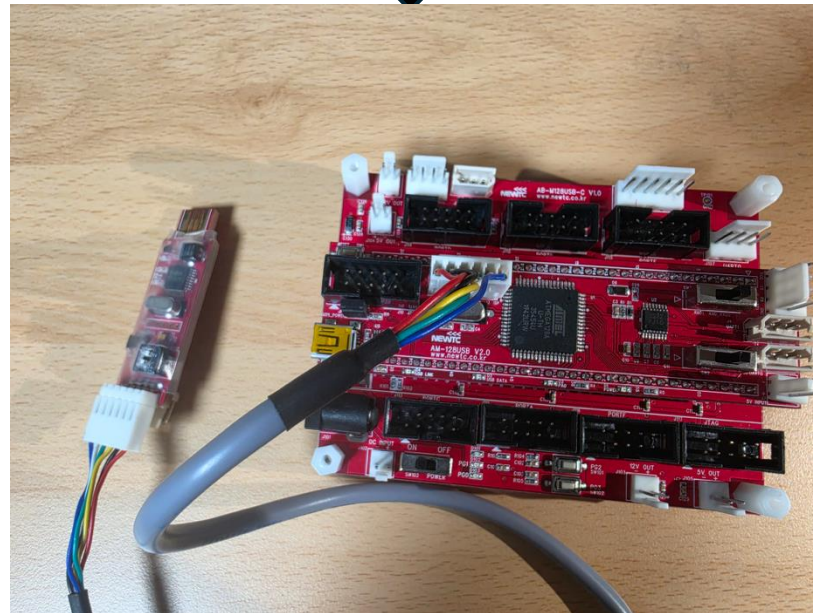

```
void callback(char* topic, byte* payload, unsigned int length) {
  // 수신된 첫 번째 문자가 '0'(Open)인지 확인 [cite: 139, 294]
  if ((char)payload[0] == '0') {
    // ATmega128의 RX(PE0) 핀으로 '0' 문자 전송 [cite: 110, 277]
    Serial.print('0');
  }
}
```



```
// UART1 수신 인터럽트
interrupt [USART1_RXC] void usart1_rx_isr(void)
{
  char data;

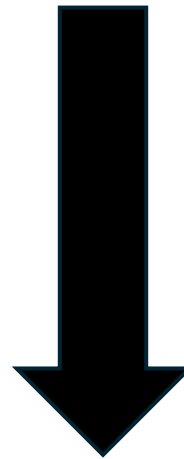
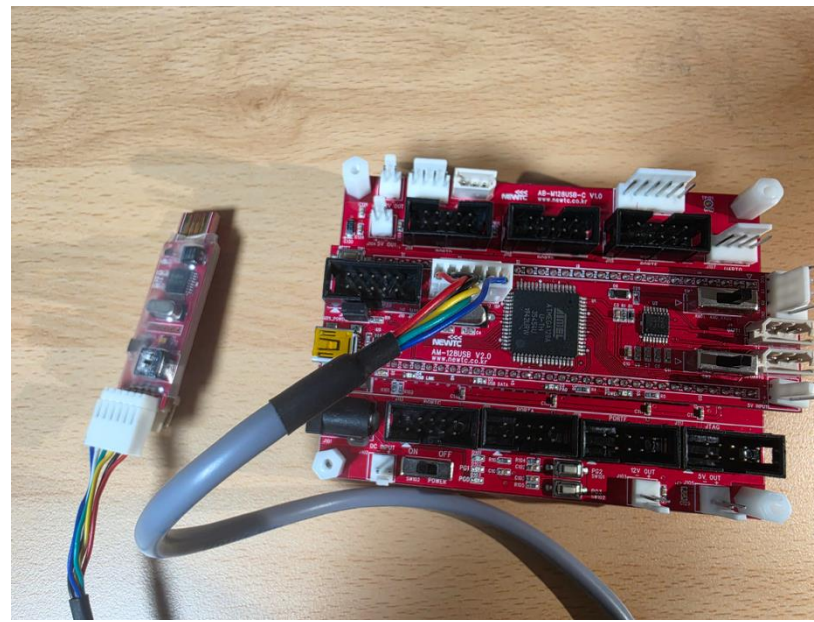
  data = UDR1;

  if(data == '0')
  {
    open_door();
  }
}
```



인터럽트가 발생하면
받은 데이터가 0인지 확인하고
0이면 선언된 open_door() 함수 호출

```
void open_door(void)
{
    OCR1AH = 0x01;
    OCR1AL = 0xF4; // OCR1A = 500, 약 90도
    delay_ms(2000);
    OCR1AH = 0x00;
    OCR1AL = 150; // 다시 잠금 위치
}
```



동작!


```

#define F_CPU 16000000UL
#include <mega128.h>
#include <delay.h>
// UART1 설정 : ESP8266 통신용
void uart1_init(void)
{
    // 9600bps 설정
    // 16MHz 기준 UBRR = 103
    UBRR1H = 0x00;
    UBRR1L = 103;
    UCSR1A = 0x00;
    UCSR1C = 0x06;
    // RX, TX, RX 인터럽트 활성화
    UCSR1B = 0x98;

}

void pwm_init(void)
{
    DDRB |= 0b00100000; // PB5(OC1A) 출력 설정
    // Fast PWM Mode 14
    // COM1A1 = 1 : OC1A 비반전 출력
    // WGM11 = 1
    TCCR1A = 0b10000010;
    // WGM13 = 1, WGM12 = 1
    // Prescaler = 64 → CS11=1, CS10=1
    TCCR1B = 0b00011011;
    ICR1H = 0x13;
    ICR1L = 0x87; // ICR1 = 4999
    OCR1AH = 0x00;
    OCR1AL = 150; // 초기 잠금 위치

}

```

```

void open_door(void)
{
    OCR1AH = 0x01;
    OCR1AL = 0xF4; // OCR1A = 500, 약 90도
    delay_ms(2000);
    OCR1AH = 0x00;
    OCR1AL = 150; // 다시 잠금 위치
}

// UART1 수신 인터럽트
interrupt [USART1_RXC] void usart1_rx_isr(void)
{
    char data;

    data = UDR1;

    if(data == '0')
    {
        open_door();
    }
}

void main(void)
{
    uart1_init();
    pwm_init();

    #asm("sei") // 전역 인터럽트 활성화

    while(1)
    {
        // 메인 루프
    }
}

```

감사합니 다